

NBA ANALYTICS

Pipeline de Engenharia de Dados

dbt Core · PostgreSQL · Python · Selenium

■ O QUE VOCÊ VAI APRENDER

■ O que é dbt e por que ele existe	■ Cada pasta e arquivo do projeto
■ Como funciona o pipeline completo	■ Regras de negócio implementadas
■■ Arquitetura de 3 camadas (staging/int/marts)	■ Testes de qualidade de dados
■ Os scrapers Python (Selenium + BBR)	■ Como falar sobre tudo isso em entrevista

PARTE 1 — O Que é dbt e Por Que Usamos

1.1 A Definição Simples

dbt significa **data build tool**. É uma ferramenta open-source que permite escrever transformações de dados em SQL puro, mas com superpoderes: dependências automáticas entre modelos, testes de qualidade, documentação gerada automaticamente e controle de versão.

Pense no dbt como o equivalente do **Git + Make + pytest** para SQL. Você escreve `SELECT`, o dbt cuida do `CREATE TABLE/VIEW`, da ordem certa de execução e dos testes.

1.2 O Problema Que o dbt Resolve

Antes do dbt, pipelines de dados em empresas eram caóticos: scripts SQL espalhados em pastas, sem saber qual rodava antes de qual, sem testes, e documentação inexistente ou desatualizada. Um analista quebrava um pipeline sem perceber porque não sabia que aquela tabela era usada em outros 5 lugares.

■ Sem dbt	■ Com dbt
SQL em arquivos <code>.sql</code> sem ordem definida	Ordem automática via grafo de dependências (DAG)
<code>CREATE TABLE IF NOT EXISTS</code> na mão	dbt materializa como <code>TABLE</code> ou <code>VIEW</code> automaticamente
Zero testes de dados	Testes declarativos: <code>unique</code> , <code>not_null</code> , <code>relationships</code>
Documentação em planilhas desatualizadas	Documentação gerada do próprio código <code>YAML</code>
'Quem usa essa tabela?' — ninguém sabe	Lineage graph mostra o impacto de cada mudança
Deploy manual e arriscado	dbt run executa só o que mudou, na ordem certa

1.3 Como o dbt Funciona (o ciclo de vida)

O dbt opera em cima de um banco de dados já existente — no nosso caso, PostgreSQL. Ele **não move dados**, ele **transforma dados que já estão no banco**. O fluxo é:

- Você escreve um arquivo `.sql` com um `SELECT`
- O dbt compila esse `SELECT` e gera o SQL final com os nomes de tabela corretos
- O dbt executa no PostgreSQL, criando `TABLE` ou `VIEW` dependendo da configuração
- O dbt registra o resultado no seu metadata e oferece docs e testes

■ **Analogia para entrevista:** dbt está para SQL assim como React está para JavaScript — você não escreve HTML na mão, você descreve o que quer e o framework gera o código otimizado. dbt não é um orquestrador (não é o Airflow), é um transformador.

1.4 dbt Core vs dbt Cloud

Existem dois produtos: **dbt Core** (open-source, linha de comando, gratuito) e **dbt Cloud** (SaaS, interface web, agendamento, CI/CD integrado, pago). Neste projeto usamos **dbt Core 1.9.10** — a versão de linha de comando,

que é o que você controla 100% e que aparece em entrevistas técnicas.

PARTE 2 — Arquitetura do Projeto

2.1 Visão Geral do Pipeline

O projeto segue o padrão **ELT** (Extract → Load → Transform), não ETL. A diferença é importante:

- **ETL** — transforma os dados ANTES de carregar no banco. Mais antigo, menos flexível.
- **ELT** — carrega os dados RAW primeiro, transforma DENTRO do banco. Padrão moderno (dbt é ELT).

Etapa	Ferramenta	O que acontece	Saída
Extract	Python + Selenium	Scraping do Basketball Reference	Arquivos CSV
Load	dbt seed	Carrega os CSVs no PostgreSQL (schema raw)	Tables raw
Transform 1	dbt staging	Limpa tipos, renomeia colunas	Views staging
Transform 2	dbt intermediate	Regras de negócio (dedup jogadores)	Views intermediate
Transform 3	dbt marts	Modelo dimensional (dim_*, fct_*)	Tables marts

2.2 A Separação em Schemas do PostgreSQL

O dbt cria schemas diferentes no PostgreSQL para cada camada. Isso é feito com a configuração **+schema** no `dbt_project.yml`. O prefixo **analytics_** é o schema padrão do profile + o sufixo configurado.

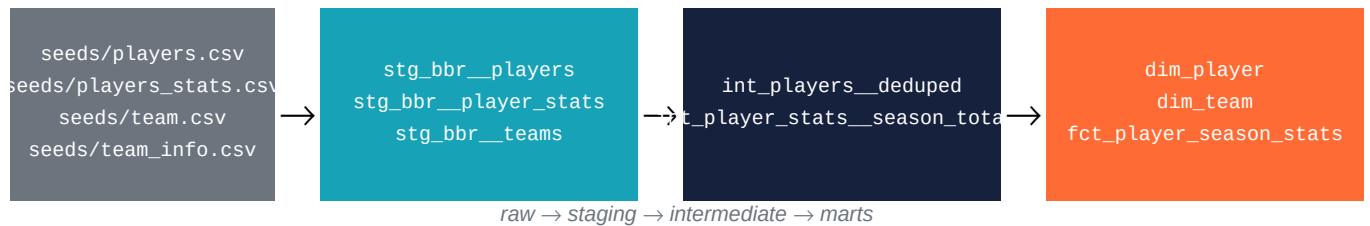
Schema no PostgreSQL	Camada dbt	Tipo de objeto	Quem usa
analytics_raw	seeds	TABLE	Só o dbt (não exponha para analistas)
analytics_staging	staging	VIEW	Engenheiros de dados, para debug
analytics_intermediate	intermediate	VIEW	Engenheiros de dados, para debug
analytics_marts	marts	TABLE	Analistas, dashboards, notebooks

■ Por que VIEW para staging/intermediate e TABLE para marts?

VIEWS não ocupam espaço em disco — elas executam a query na hora que você consulta. Para camadas intermediárias que mudam todo dia, isso é ótimo. TABLEs persistem os dados e são mais rápidas para consultas analíticas dos usuários finais. Marts são consultados por analistas que não devem esperar a query rodar do zero toda vez.

2.3 O DAG — Grafo de Dependências

O coração do dbt é o **DAG (Directed Acyclic Graph)**. Quando você usa `{{ ref('nome_modelo') }}` dentro de um arquivo SQL, o dbt entende que aquele modelo depende do outro e garante a ordem correta de execução. Nunca vai tentar criar `fct_player_season_stats` antes de `dim_player` estar pronta.



■ Pergunta de entrevista: O que é um DAG no contexto de pipelines de dados?

DAG é um Grafo Acíclico Dirigido — uma estrutura onde cada nó é uma tarefa e as arestas representam dependências. 'Acíclico' significa que não há loops: A depende de B, B depende de C, mas C não pode depender de A. No dbt, o DAG é construído automaticamente a partir das funções `ref()` nos modelos SQL. Isso garante ordem de execução correta e permite paralelizar modelos independentes.

PARTE 3 — Cada Arquivo e Pasta Explicado

3.1 dbt_project.yml — O Arquivo de Configuração Central

É o arquivo que diz ao dbt *quem você é*. Fica na raiz do projeto e define: nome do projeto, qual profile usar, onde ficam os modelos/seeds/macros, e configurações globais de materialização e schema.

```
name: nba_analytics          # nome do projeto
profile: nba_analytics       # aponta para .dbt/profiles.yml

seed-paths: ["seeds"]       # onde ficam os CSVs
model-paths: ["models"]     # onde ficam os .sql
macro-paths: ["macros"]     # onde ficam as macros Jinja

seeds:
  nba_analytics:
    +schema: raw             # seeds vão para schema analytics_raw
    +quote_columns: true     # preserva maiúsculas nos nomes de colunas

models:
  nba_analytics:
    staging:
      +materialized: view    # staging = VIEW (sem custo de armazenamento)
      +schema: staging       # analytics_staging
    intermediate:
      +materialized: view
      +schema: intermediate
    marts:
      +materialized: table   # marts = TABLE (rápido para analistas)
      +schema: marts
```

■ ■ **quote_columns: true** — Basketball Reference usa colunas como Player, Team com letra maiúscula. No PostgreSQL, nomes sem aspas são convertidos para minúsculo. quote_columns: true força o dbt a criar a coluna como "Player" (com aspas) preservando o case original.

3.2 .dbt/profiles.yml — Credenciais de Conexão

O profiles.yml diz ao dbt *como conectar ao banco de dados*. É o arquivo mais sensível do projeto — nunca deve ir para o Git com credenciais reais. Por isso está no .gitignore e usamos **env_var()** para ler do ambiente.

```
nba_analytics:                # mesmo nome que 'profile' no dbt_project.yml
  target: dev
  outputs:
    dev:
      type: postgres
      host: "{{ env_var('DBT_HOST', 'localhost') }}"
      port: "{{ env_var('DBT_PORT', '5432') | int }}"
      dbname: "{{ env_var('DBT_DBNAME', 'nba') }}"
      user: "{{ env_var('DBT_USER', 'postgres') }}"
      password: "{{ env_var('DBT_PASSWORD', 'postgres') }}"
      schema: analytics        # schema padrão (prefixo para outros schemas)
      threads: 4               # quantos modelos rodam em paralelo
```

A sintaxe `{{ env_var('DBT_HOST', 'localhost') }}` é **Jinja2**, o sistema de template que o dbt usa. Ela lê a variável de ambiente `DBT_HOST`; se não existir, usa `'localhost'` como fallback. Isso permite que em produção você defina as variáveis de ambiente sem tocar no código.

3.3 seeds/ — A Camada Raw (Dados Brutos)

Seeds são arquivos CSV que o dbt carrega no banco com **dbt seed**. Eles representam a camada mais crua dos dados — exatamente como vieram da fonte, sem transformação. No nosso caso, são os arquivos gerados pelos scrapers Python.

Arquivo	Origem	Linhas (aprox.)	Schema carregado
players.csv	BBR per-game page (roster)	735	analytics_raw.players
players_stats.csv	BBR per-game page (stats)	735	analytics_raw.players_stats
team.csv	BBR teams page	87	analytics_raw.team
team_info.csv	Estático (mantido à mão)	30	analytics_raw.team_info

• Por que usar dbt seed em vez de carregar direto no banco?

- O seed entra no DAG do dbt — os modelos que dependem dele sabem que precisam esperar o seed terminar
- O schema.yml do seed documenta e testa os dados da camada raw
- Se o CSV mudar, dbt seed recria a tabela automaticamente

seeds/schema.yml — Documento os seeds com descrições e testes de qualidade. Note que os testes de `not_null` ficam no staging (não no seed) por um motivo técnico: o PostgreSQL com `quote_columns` cria a coluna como **"Player"** (case-sensitive) mas o dbt gera o teste SQL como **Player** (sem aspas, vai para lowercase). Os testes ficam na camada seguinte onde os nomes já foram normalizados.

3.4 macros/ — Funções Reutilizáveis em Jinja

Macros são funções Jinja2 que você pode chamar de qualquer modelo dbt. São o equivalente de funções Python no mundo dbt — evitam repetição de lógica.

macros/generate_surrogate_key.sql

Gera uma **chave substituta (surrogate key)** usando MD5 de uma lista de colunas. Uma chave substituta é um ID artificial que não depende dos dados reais — é mais estável que usar o nome do jogador diretamente como chave primária.

```
{% macro generate_surrogate_key(field_list) %}
  md5(
    {% for field in field_list %}
      coalesce(cast({{ field }} as text), '__NULL__')
    {% if not loop.last %} || '|' || {% endif %}
    {% endfor %}
  )
{% endmacro %}
```

Uso: `{{ generate_surrogate_key(['player_name']) }}` → gera MD5('LeBron James') → um hash de 32 caracteres que identifica o jogador de forma única. Se o mesmo jogador aparecer em datasets de temporadas diferentes, a chave continua estável porque é derivada do nome.

■ Por que não usar o nome do jogador diretamente como PK?

Porque nomes mudam (correção de grafia, nomes artísticos), e joins com strings são mais lentos do que joins com hashes de tamanho fixo. Em produção com milhões de registros, isso faz diferença mensurável.

PARTE 4 — Staging Layer (Camada de Limpeza)

4.1 O Que É e Por Que Existe

A staging layer tem **uma única responsabilidade**: pegar os dados brutos do seed e limpá-los — sem lógica de negócio, sem joins entre fontes diferentes, sem cálculos. Apenas: renomear colunas, corrigir tipos de dados, filtrar linhas inválidas.

O princípio fundamental é: **se a fonte mudar, só a staging precisa mudar**. Todos os modelos downstream confiam nos contratos da staging — nomes de colunas e tipos de dados estáveis.

4.2 Convenção de Nomenclatura: stg_bbr__entidade.sql

O nome dos arquivos segue a convenção: **stg_[fonte]__[entidade].sql**

- **stg** = staging layer
- **bbr** = Basketball Reference (a fonte dos dados)
- **__** (dois underscores) = separador fonte/entidade (convenção dbt oficial)
- **players, player_stats, teams** = a entidade/domínio

Essa convenção é importante porque quando você tem múltiplas fontes (ex: stg_salesforce__accounts, stg_stripe__payments), fica imediatamente claro de onde cada dado veio.

4.3 stg_bbr__players.sql — Análise Linha a Linha

```
with source as (  
  select * from {{ ref('players') }} -- ref() = depende do seed 'players'  
)  
  
cleaned as (  
  select  
    trim("Player")           as player_name, -- remove espaços  
    upper(trim("Team"))      as team_abbr,  -- padroniza maiúsculo  
    upper(trim("Pos"))       as position,   -- G, F, C...  
    nullif(trim("Age)::text", '')::integer as age      -- string → int  
  from source  
  where trim("Player") != 'Player'          -- remove linhas de cabeçalho repetidas  
)  
  
select * from cleaned
```

Padrões que aparecem aqui e por que existem:

Padrão	Exemplo	Motivo
{{ ref() }}	ref('players')	Cria dependência no DAG. dbt garante que o seed existe antes de rodar este modelo.
trim()	trim("Player")	BBR às vezes inclui espaços em branco no início/fim de strings.
upper()	upper(trim("Team"))	Padroniza para comparações case-insensitive downstream.

<code>nullif()</code>	<code>nullif(trim(x), '')</code>	Converte string vazia em NULL, que é o valor correto semanticamente.
<code>::integer</code>	<code>trim(x)::integer</code>	Cast explícito de tipo — seeds carregam tudo como texto, precisamos tipar.
<code>where != 'Player'</code>	<code>trim("Player") != 'Player'</code>	BBR repete a linha de cabeçalho a cada ~100 linhas na tabela HTML paginada.
Colunas entre aspas	<code>"Player", "Team"</code>	Os nomes no PostgreSQL são case-sensitive quando criados com <code>quote_columns: true</code> .

4.4 stg_bbr__player_stats.sql — O Desafio das Colunas Especiais

Basketball Reference usa nomes de colunas com caracteres que são inválidos em SQL: **FG%** (símbolo de porcentagem), **3P** (começa com número), **eFG%** (mistura tudo). Esses nomes foram renomeados no scraper Python antes de salvar o CSV para que o dbt seed pudesse carregar sem erros.

Nome original BBR	Nome no CSV/seed	Motivo da renomeação
FG%	fg_pct	% é operador em SQL
3P	three_p	Nomes não podem começar com dígito em SQL
3PA	three_pa	Mesmo motivo
3P%	three_p_pct	Ambos os problemas
2P	two_p	Começa com dígito
2PA	two_pa	Começa com dígito
2P%	two_p_pct	Ambos os problemas
eFG%	efg_pct	% é operador em SQL
FT%	ft_pct	% é operador em SQL
W/L%	wl_pct	/ e % são operadores em SQL (no team.csv)

4.5 stg_bbr__teams.sql — Dados Históricos de Franquias

O BBR lista a história de cada franquia com múltiplas eras (ex: Oklahoma City Thunder tem uma era como Seattle SuperSonics). A staging não filtra isso — ela limpa os tipos e repassa todas as linhas. A lógica de filtrar só a era atual fica em dim_team (marts).

■ **Princípio de design:** a staging não sabe o que é 'correto' do ponto de vista de negócio — ela só sabe o que está na fonte. Decisões de negócio ('quero só a era atual') ficam nas camadas downstream onde são explícitas e documentadas.

PARTE 5 — Intermediate Layer (Regras de Negócio)

5.1 O Que É a Camada Intermediate

A intermediate layer existe para lógica de negócio que é **reutilizada por múltiplos marts** mas não pertence à staging (que não tem lógica) e não pertence diretamente ao mart (que ficaria muito complexo). É o equivalente de uma função auxiliar no código Python.

Convenção de nome: `int_[domínio]__[propósito].sql`

Exemplos: `int_players__deduped`, `int_player_stats__season_totals`

5.2 O Problema do Jogador Trocado — A Regra de Negócio Principal

Este é o exemplo perfeito do tipo de problema que aparece em dados reais e que entrevistadores adoram explorar. Quando um jogador é trocado de time durante a temporada, o Basketball Reference registra **múltiplas linhas** para ele:

Player	Team	G	PTS
Dennis Schröder	BRK	12	12.1
Dennis Schröder	GSW	28	9.3
Dennis Schröder	DET	31	14.2
Dennis Schröder	3TM	71	11.8 ← linha agregada (3 times)

Se você somar todos os jogos de Dennis Schröder, vai contar 142 jogos (12+28+31+71) quando a temporada tem 82 jogos no máximo. O registro **3TM** (3 times) é o agregado correto — 71 jogos, média ponderada de pontos. **Histórico:** o BBR usava 'TOT' para isso, mas a partir de 2024-25 passou a usar '2TM', '3TM', '4TM' conforme o número de times.

A solução implementada em `int_players__deduped.sql`:

```
traded_players as (
  -- Identifica jogadores que têm linha de agregado
  select distinct player_name
  from players
  where team_abbr = 'TOT'           -- formato antigo
     or team_abbr ~ '^\\d+TM$'      -- formato novo: 2TM, 3TM, 4TM...
),

deduped as (
  select p.*
  from players p
  left join traded_players t using (player_name)
  where
    t.player_name is null           -- não foi trocado: mantém a linha única
    or p.team_abbr = 'TOT'         -- foi trocado: mantém só o agregado
    or p.team_abbr ~ '^\\d+TM$'
)
```

■ Pergunta de entrevista: Como você lidaria com dados duplicados de um jogador trocado de time?

No Basketball Reference, cada jogador trocado tem N+1 linhas: uma por time e uma com o total agregado (TOT/NTM). Para análise de temporada completa, o correto é manter apenas o agregado, que já tem a média ponderada pelo número de jogos em cada time. Implementei isso com um LEFT JOIN para identificar jogadores com linha de agregado e filtrei para manter apenas essa linha. O desafio extra foi que o BBR mudou a nomenclatura de 'TOT' para '2TM/3TM' na temporada atual — resolvi com regex '`^\d+TM$`' para ser compatível com ambos os formatos.

PARTE 6 — Marts Layer (Modelo Dimensional)

6.1 Modelagem Dimensional — dim_ e fct_

A camada marts implementa o **modelo dimensional** (também chamado de Star Schema), o padrão consagrado para data warehouses analíticos. Foi formalizado por Ralph Kimball nos anos 90 e continua sendo o padrão dominante em 2025.

Tipo	Prefixo	O que contém	Características
Dimension	dim_	Atributos descritivos das entidades	Muda raramente, tem surrogate key, enriquece análises
Fact	fct_	Eventos/métricas mensuráveis	Cresce constantemente, tem FKs para dimensions, é o grain da análise

6.2 dim_player.sql — Dimensão do Jogador

Uma linha por jogador ativo na temporada. Contém atributos que descrevem o jogador: nome, posição, idade, time atual, conferência, divisão. Não contém estatísticas — essas ficam na fct_.

Lógica de resolução do time atual para jogadores trocados:

```
-- Jogadores trocados têm team_abbr = '3TM' na tabela deduped.
-- Precisamos saber em qual time eles TERMINARAM a temporada.
-- Buscamos o time com mais jogos nas linhas por-time (excluindo o agregado).

last_team as (
  select player_name, team_abbr as last_team_abbr
  from (
    select player_name, team_abbr, games_played,
           row_number() over (
             partition by player_name
             order by games_played desc -- time com mais jogos = último
           ) as rn
    from stg_bbr__player_stats
    where team_abbr != 'TOT'
         and team_abbr !~ '^\\d+TM$' -- exclui as linhas de agregado
  ) ranked
  where rn = 1
)
```

6.3 dim_team.sql — Dimensão do Time

30 linhas — uma por franquia ativa da NBA. Combina o seed estático team_info (conferência, divisão — dados que o BBR não fornece diretamente) com os dados históricos de vitórias/derrotas do BBR teams page.

Desafio: múltiplas eras históricas por franquia

O BBR lista cada era de uma franquia separadamente. Por exemplo, o Boston Celtics tem múltiplas linhas com season_to = '2025-26' (era de 1946-47 até hoje, etc.). A solução foi usar ROW_NUMBER() para selecionar apenas a linha com mais jogos disputados (o record all-time, que inclui toda a história).

6.4 fct_player_season_stats.sql — A Tabela Fato

Grain (granularidade): uma linha por jogador por temporada. Este é o conceito mais importante do modelo dimensional — cada linha da tabela fato representa exatamente um evento/medição de nível específico.

Coluna	Tipo	O que representa
fact_key	TEXT (MD5)	Surrogate key da linha fato
player_key	TEXT (MD5)	FK → dim_player
team_key	TEXT (MD5)	FK → dim_team
season	TEXT	Degenerate dimension: '2024-25'
games_played	INTEGER	Fact: total de jogos
points_per_game	NUMERIC	Fact: média de pontos
fg_pct	NUMERIC	Fact: porcentagem de arremessos
... (25 colunas)	NUMERIC	Todas as stats per-game do BBR

■ **Degenerate Dimension (season):** O campo 'season' é uma dimensão degenerada — um atributo descritivo que não vale a pena colocar numa tabela dim_season separada porque só tem um valor neste projeto (2024-25). Em um data warehouse com múltiplas temporadas, existiria uma dim_season com atributos como playoffs_year, all_star_location, etc.

■ Pergunta de entrevista: O que é Star Schema e por que você escolheu esse modelo?

Star Schema é um modelo dimensional onde uma tabela fato central se conecta a múltiplas tabelas dimensão por chaves estrangeiras. A forma do diagrama ER lembra uma estrela — a fato no centro, as dims nas pontas. Escolhi esse modelo porque (1) facilita queries analíticas intuitivas — analistas fazem JOIN fct com dim e groupam por qualquer atributo dimensional; (2) queries de agregação são rápidas porque a tabela fato tem só números e FKs; (3) é o padrão que ferramentas de BI como Metabase e Looker entendem nativamente. A alternativa seria um modelo 3NF (normalização completa), que é ótimo para sistemas transacionais mas ruim para analytics porque exige muitos JOINS.

PARTE 7 — Testes de Qualidade de Dados

7.1 Por Que Testes em Dados São Críticos

Diferente de código de software, onde um bug geralmente causa um erro imediato, bugs em dados são silenciosos. Um dashboard mostrando o número errado de pontos por jogo pode enganar um analista por semanas antes de alguém perceber. Testes de dados são a única forma de detectar isso automaticamente.

O dbt oferece **testes genéricos** que você declara em YAML — não precisa escrever SQL. O dbt gera o SQL de teste automaticamente.

7.2 Os Quatro Tipos de Teste Usados no Projeto

Teste	O que valida	Exemplo no projeto
unique	Nenhum valor se repete na coluna	player_key em dim_player deve ser único
not_null	Nenhum valor nulo na coluna	player_name em todas as tabelas
accepted_values	Coluna só contém valores de uma lista	conference só aceita 'East' ou 'West'
relationships	FK existe na tabela referenciada (integridade referencial)	player_key em fct_player_season_stats deve existir em dim_player

7.3 Como Declarar um Teste (schema YAML)

```
# models/marts/_marts__models.yml
models:
  - name: dim_player
    columns:
      - name: player_key
        data_tests:
          - unique      # ← dbt gera: SELECT count(*) WHERE count > 1
          - not_null    # ← dbt gera: SELECT count(*) WHERE player_key IS NULL

      - name: conference
        data_tests:
          - accepted_values:
              values: ['East', 'West'] # falha se encontrar outro valor

  - name: fct_player_season_stats
    columns:
      - name: player_key
        data_tests:
          - relationships:
              to: ref('dim_player') # verifica que toda FK existe na dim
              field: player_key
```

7.4 Rodando os Testes

```
dbt test --profiles-dir .dbt          # todos os testes
dbt test --profiles-dir .dbt --select dim_player # só um modelo
dbt test --profiles-dir .dbt --select test_type:unique # só os unique
```

No nosso projeto: **26 testes passando**. Cada vez que os dados são atualizados (novo scraping), os testes são re-executados para garantir que nada quebrou.

■ Pergunta de entrevista: Como você garante qualidade de dados no seu pipeline?

No dbt implemento testes declarativos em YAML para cada camada: `unique` e `not_null` nas chaves primárias, `accepted_values` para enums (conferência Leste/Oeste), e `relationships` para integridade referencial entre `facts` e `dims`. Isso captura 80% dos problemas comuns de qualidade. Para dados de produção adicionaria também testes de volume (número de linhas dentro de um range esperado) e testes de freshness (dados não podem ter mais de X horas). Em pipelines mais maduros usaria ferramentas como Great Expectations ou Soda Core para testes mais sofisticados.

PARTE 8 — Scrapers Python (src/scraping/)

8.1 Por Que Selenium e Não requests

A primeira tentativa natural é fazer `requests.get(url)` para pegar o HTML da página. O Basketball Reference retorna **HTTP 403 Forbidden** para qualquer cliente que não pareça um navegador real. Eles detectam pelo User-Agent e pelo comportamento de não executar JavaScript.

Selenium resolve isso ao controlar um navegador real (Chromium) — o site não consegue distinguir do acesso humano. A desvantagem é lentidão (~8 segundos por página) e dependência de um ambiente com GUI (resolvido com modo headless).

8.2 Estrutura do Módulo src/scraping/

Arquivo	Função
common/browser.py	Configura o Selenium: caminhos do Chromium snap, modo headless, sleep de 8s
common/parsing.py	Função <code>uncomment_tables()</code> : BBR esconde tabelas em comentários HTML
players.py	Extrai roster (Player, Age, Team, Pos) → <code>seeds/players.csv</code>
stats.py	Extrai stats completas → <code>seeds/players_stats.csv</code> (com renomeações)
teams.py	Extrai histórico de franquias → <code>seeds/team.csv</code>
contracts.py	Extrai contratos → <code>seeds/contracts.csv</code> (armazenado para uso futuro)
run_all.py	Orquestrador: roda todos os scrapers em sequência com log de resultado

8.3 O Truque das Tabelas em Comentário HTML

BBR esconde as tabelas de dados dentro de comentários HTML para dificultar scraping: `<!-- <table id='per_game_stats'>...</table> -->`. BeautifulSoup ignora comentários por padrão — a tabela simplesmente não aparece.

```
def uncomment_tables(soup):
    from bs4 import Comment
    for c in soup.find_all(string=lambda t: isinstance(t, Comment)):
        inner = BeautifulSoup(str(c), 'lxml') # parseia o conteúdo do comentário
        if inner.find('table'):                # se tiver tabela dentro
            c.replace_with(inner)              # substitui o comentário pela tabela
    return soup

# ATENÇÃO: BeautifulSoup 4.13 quebrou soup.append() — retorna lista vazia.
# A forma correta é c.replace_with(), que funciona em todas as versões.
```

8.4 Configuração do Chromium no WSL (Ubuntu Snap)

No Windows Subsystem for Linux (WSL) com Ubuntu, o Chromium é instalado via Snap e seus executáveis ficam em caminhos não-padrão que o webdriver-manager não encontra.

```
CHROME_BINARY    = '/usr/bin/chromium-browser'      # o navegador
CHROMEDRIVER_PATH = '/snap/bin/chromium.chromedriver' # o driver

opts = Options()
opts.binary_location = CHROME_BINARY
opts.add_argument('--headless=new')      # sem janela gráfica
opts.add_argument('--no-sandbox')        # necessário no WSL
service = Service(executable_path=CHROMEDRIVER_PATH)
driver = webdriver.Chrome(service=service, options=opts)
```

PARTE 9 — Como Falar Sobre o Projeto em Entrevista

9.1 O Pitch de 2 Minutos (Elevator Pitch)

"Construí um pipeline de dados end-to-end para análise de estatísticas da NBA. Fiz scraping automatizado do Basketball Reference com Python e Selenium, porque o site bloqueia requests diretos. Os dados são carregados no PostgreSQL e transformados com dbt Core seguindo arquitetura em três camadas: staging para limpeza, intermediate para regras de negócio como de-duplicação de jogadores trocados, e marts com modelo dimensional (Star Schema) para análise. O projeto tem 26 testes automatizados de qualidade de dados e documentação gerada automaticamente no YAML."

9.2 Perguntas Técnicas — Nível Pleno

■ Pergunta de entrevista: Qual a diferença entre ETL e ELT?

ETL (Extract-Transform-Load) transforma os dados antes de carregar no destino final — era necessário quando armazenamento era caro. ELT (Extract-Load-Transform) carrega os dados brutos primeiro e transforma dentro do data warehouse — possível com o barateamento de armazenamento e aumento do poder computacional dos bancos modernos. dbt é fundamentalmente uma ferramenta ELT: os dados chegam no PostgreSQL via seed, e o dbt transforma dentro do banco.

■ Pergunta de entrevista: O que é materialização no dbt? Quando usar VIEW vs TABLE?

Materialização é como o dbt persiste o resultado de um modelo no banco. VIEW não armazena dados — executa a query na hora da consulta, sem custo de storage, mas pode ser lenta se consultada frequentemente. TABLE persiste os dados fisicamente — consulta rápida, mas ocupa storage e precisa ser reconstruída a cada run. No projeto: staging e intermediate são VIEW (mudam com frequência, raramente consultadas diretamente), marts são TABLE (consultadas por analistas, velocidade importa). Existe também INCREMENTAL (só processa novos registros) e EPHEMERAL (CTE inline, sem objeto no banco).

■ Pergunta de entrevista: Como funciona o `{{ ref() }}` no dbt?

`ref()` é a função central do dbt. Quando você escreve `ref('stg_bbr__players')`, o dbt (1) resolve para o nome completo da tabela/view no banco (schema + nome), (2) registra uma dependência no DAG — este modelo depende de `stg_bbr__players`, garantindo que ele exista antes de tentar criar o atual. É o que permite ao dbt calcular a ordem correta de execução e paralelizar modelos independentes. `source()` serve para referenciar tabelas externas não gerenciadas pelo dbt.

■ Pergunta de entrevista: O que é um surrogate key e por que você usou MD5?

Surrogate key é uma chave primária artificial — gerada pelo sistema, não derivada dos dados reais. Usamos `MD5(player_name)` porque: (1) é determinístico — o mesmo nome sempre gera o mesmo hash, permitindo joins sem precisar de um sequence ou UUID centralizado; (2) é compacto — 32 chars vs nomes de comprimento variável; (3) é estável mesmo que a lógica de geração migre entre sistemas. A alternativa seria usar `SERIAL/IDENTITY` no PostgreSQL, mas isso complica joins entre modelos que não passam pela mesma transação.

9.3 Perguntas Técnicas — Nível Sênior

■ Pergunta de entrevista: Como você escalaria esse pipeline para múltiplas temporadas?

Adicionaria uma coluna `season` em todos os seeds e modelos, e usaria materialização `INCREMENTAL` no dbt para processar apenas dados novos. No staging, adicionaria um filtro `{{ this.filters }}` para ler só CSVs da temporada nova. Na `fct_`, a chave seria `(player_name, season)` em vez de só `player_name`. Para `dim_player`, implementaria SCD Type 2 (Slowly Changing Dimension) para rastrear mudanças de time entre temporadas — cada mudança gera uma nova versão do registro com `effective_from/effective_to`.

■ Pergunta de entrevista: Quais problemas de qualidade de dados você encontrou e como resolveu?

Três principais: (1) BBR repete a linha de cabeçalho a cada página — filtrei com `WHERE player != 'Player'` no staging. (2) Jogadores trocados aparecem múltiplas vezes — resolvi com CTE de de-duplicação na camada intermediate usando o padrão NTM. (3) BBR mudou nomenclatura de 'TOT' para '2TM/3TM' na temporada atual — descoberto apenas quando os testes `unique` falharam com 81 duplicatas. Resolvi com regex `^\d+TM$` para capturar ambos os formatos. Isso mostra a importância dos testes — sem eles, os dados estariam errados silenciosamente.

■ Pergunta de entrevista: Como você organizaria CI/CD para um pipeline dbt em produção?

No GitHub Actions: (1) PR → dbt compile (valida sintaxe) + dbt test em um schema de CI isolado com dados de sample. (2) Merge em main → dbt run + dbt test completo em staging environment. (3) Aprovação manual → deploy em produção. Usaria dbt slim CI (`--state flag`) para rodar apenas modelos que mudaram, reduzindo tempo de execução. Para o agendamento, integraria com Airflow ou Dagster em vez de cron — eles gerenciam retries, alertas e dependências entre pipelines.

■ Pergunta de entrevista: Por que PostgreSQL em vez de DuckDB ou Snowflake?

DuckDB seria mais simples para desenvolvimento local — não precisa de servidor, lê CSVs nativamente. Escolhi PostgreSQL para demonstrar setup de produção: configuração de conexão com credentials via `env_var`, separação de schemas, compatibilidade com ferramentas BI padrão de mercado. Em produção real com volume, usaria BigQuery ou Snowflake — o dbt suporta ambos com apenas mudança no `profiles.yml` (`type: bigquery` ou `type: snowflake`).

9.4 Perguntas Comportamentais Sobre o Projeto

■ Pergunta de entrevista: Qual foi o maior desafio técnico do projeto?

A detecção da mudança de nomenclatura do BBR de 'TOT' para 'NTM' durante o desenvolvimento. Os modelos compilavam e rodavam sem erro — só os testes de unicidade revelaram 81 duplicatas inesperadas. Investiguei consultando o banco diretamente, descobri o padrão '3TM' e atualizei a regex de detecção. Isso reforçou a importância de testes automatizados: sem eles, o dashboard mostraria estatísticas infladas para ~80 jogadores e ninguém perceberia.

■ Pergunta de entrevista: Por que você escolheu essa estrutura de camadas?

A separação staging/intermediate/marts vem do dbt best practices guide e da experiência coletiva da comunidade. O princípio central é: cada camada tem uma responsabilidade única e imutável. Staging só limpa, intermediate só aplica regras de negócio reutilizáveis, marts expõem para o usuário final. Se o Basketball Reference mudar o nome de uma coluna, só staging precisa mudar — intermediate e marts ficam intactos. Sem essa separação, uma mudança na fonte pode quebrar múltiplos modelos de forma não-óbvia.

9.5 Red Flags Para Evitar na Entrevista

■ O que não dizer	■ O que dizer no lugar
'Eu segui um tutorial'	'Enfrentei e resolvi esses problemas específicos...'
'Não sei por que funciona assim'	'A razão para essa escolha foi...' (explique o trade-off)
'dbt é tipo um ORM'	'dbt é um transformador SQL com gestão de dependências e testes'
'Não testei'	'Tenho 26 testes cobrindo unicidade, not_null e integridade referencial'
'É um projeto pessoal simples'	'Implementei padrões de produção: separação de ambientes, credenciais via env_va

PARTE 10 — Glossário e Próximos Passos

10.1 Glossário de Termos

Termo	Definição
DAG	Directed Acyclic Graph — grafo de dependências sem ciclos. No dbt, define a ordem de execução.
ELT	Extract-Load-Transform — dados são carregados brutos no banco e transformados lá dentro.
Grain	A granularidade de uma tabela fato — o que cada linha representa (ex: 1 linha = 1 jogador por temporada).
Jinja2	Sistema de templates Python usado pelo dbt para lógica em SQL (<code>{{ ref() }}</code> , <code>{% if %}</code> , etc.)
Lineage	O mapa de dependências entre modelos — mostra de onde os dados vêm e para onde vão.
Macro	Função Jinja reutilizável no dbt — como <code>generate_surrogate_key()</code> .
Materialização	Como o dbt persiste um modelo: view, table, incremental ou ephemeral.
Profile	Arquivo de configuração com credenciais de conexão ao banco (<code>.dbt/profiles.yml</code>).
ref()	Função dbt que cria dependência entre modelos e resolve o nome completo da tabela.
Schema	Namespace no PostgreSQL que agrupa tabelas (<code>analytics_raw</code> , <code>analytics_marts</code> , etc.)
Seed	Arquivo CSV carregado no banco pelo dbt seed — usado para dados de referência estáticos.
SCD Type 2	Slowly Changing Dimension — técnica para rastrear histórico de mudanças em dimensões.
Surrogate Key	ID artificial gerado pelo sistema (MD5, UUID) — independente dos dados de negócio.
Star Schema	Modelo dimensional com tabela fato central e tabelas dim nas bordas.
TOT / NTM	Abreviações BBR para jogadores trocados: TOT (legado) = Total, NTM = N Times.

10.2 Próximos Passos para Evoluir o Projeto

Evolução	Dificuldade	O que demonstra
Adicionar temporadas anteriores (SCD Type 2)	■■■	Modelagem temporal avançada
Agendamento com Airflow ou Prefect	■■■	Orquestração de pipelines
Dashboard com Metabase ou Superset	■■	Entrega de valor para o usuário final
CI/CD com GitHub Actions (dbt slim CI)	■■■	Maturidade de engenharia
Modelo incremental (só processa dados novos)	■■■	Performance em escala
API de contratos para análise salarial	■■	Expansão de domínio
dbt docs serve + publicação online	■	Documentação acessível para recrutadores
Migrar para DuckDB (mais simples de demonstrar)	■	Portabilidade
Testes customizados com macros dbt	■■■	Qualidade de dados avançada

10.3 Recursos Para Estudar Mais

- **Documentação oficial dbt:** docs.getdbt.com — a mais completa e bem escrita
- **dbt Learn:** courses.getdbt.com — curso gratuito oficial
- **Livro The Data Warehouse Toolkit** (Kimball) — fundamentos do modelo dimensional
- **dbt Style Guide:** github.com/dbt-labs/corp/blob/main/dbt_style_guide.md
- **Comunidade dbt Slack:** getdbt.com/community — 50k+ membros, muito ativo

NBA Analytics Portfolio — Projeto completo disponível no GitHub
dbt Core 1.9 · PostgreSQL 17 · Python 3.12 · Selenium 4